

Cycle 1

PROGRAM

1) LEXICAL ANALYSIS

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

int isKeyword(const char *word) {
    const char *key[] = {
        "auto", "break", "case", "char", "const", "continue", "default",
        "do", "double", "else", "enum", "extern", "float", "for",
        "goto", "if", "int", "long", "register", "return", "short",
        "signed", "sizeof", "static", "struct", "switch", "typedef",
        "union", "unsigned", "void", "volatile", "while", "main", "printf", "scanf"
    };
    int totalKeywords = sizeof(key) / sizeof(key[0]);
    for (int i = 0; i < totalKeywords; i++) {
        if (strcmp(key[i], word) == 0)
            return 1;
    }
    return 0;
}

int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '=');
}

int isDelimiter(char c) {
    return (c == '(' || c == ')' || c == '{' || c == '}' || c == ';' || c == ',');
}

void removeComments(FILE *src, FILE *dest) {
    int ch, next;
    while ((ch = fgetc(src)) != EOF) {
        if (ch == '/') {
            next = fgetc(src);
            if (next == '*') {
                while ((ch = fgetc(src)) != EOF) {
                    if (ch == '*' && fgetc(src) == '/') break;
                }
            } else if (next == '/') {

```

```

        while ((ch = fgetc(src)) != '\n' && ch != EOF);
    } else {
        fputc(ch, dest);
        fputc(next, dest);
    }
} else {
    fputc(ch, dest);
}
}
}

int main()
{
    FILE *src=fopen("sample.txt","r");
    FILE *temp=fopen("cleaned.txt","w+");

    if (src==NULL||temp==NULL)
    {
        printf("error opening file.\n");
        return 1;
    }
    removeComments(src,temp);
    fclose(src);
    rewind(temp);
    int ch;
    char buffer[100];
    while((ch=fgetc(temp))!=EOF)
    {
        if(isspace(ch))
            continue;
        if(isOperator(ch))
        {
            printf("[operator] %c\n",ch);
        }
        else if(isDelimiter(ch))
        {
            printf("[Delimiter] %c\n",ch);
        }
        else if(isdigit(ch))
        {
            int i=0,hasDot=0;
            buffer[i++]=ch;
            while((ch=fgetc(temp))!=EOF&&(isdigit(ch)||ch=='.'))
            {
                if(ch=='.') hasDot=1;

```

```

        buffer[i++]=ch;
    }
    buffer[i++] = "";
    buffer[i] = '\0';
    printf("[String Literal] %s\n", buffer);
}
else if (isalpha(ch) || ch == '_') {
    int i = 0;
    buffer[i++] = ch;
    while ((ch = fgetc(temp)) != EOF && (isalnum(ch) || ch == '_')) {
        buffer[i++] = ch;
    }
    buffer[i] = '\0';
    ungetc(ch, temp);
    if (isKeyword(buffer))
        printf("[Keyword] %s\n", buffer);
    else
        printf("[Identifier] %s\n", buffer);
}
else {
    printf("[Unknown] %c\n", ch);
}
}

fclose(temp);
return 0;
}

```

output:

```

[Keyword] int
[Keyword] main
[Delimiter] (
[Delimiter] )
[Delimiter] {
[Keyword] float
[Identifier] x
[operator] =
[String Literal] 3.14"
[Keyword] int
[Identifier] y
[operator] =
[String Literal] 1000"
[Keyword] printf

```

```
[Delimiter] (  
[Unknown] "  
[Identifier] Hi  
[Unknown] "  
[Delimiter] )  
[Delimiter] ;  
[Delimiter] }
```

2) EPSILON CLOSURE

```
#include<stdio.h>
#include<string.h>

char result[20][20],copy[3],states[20][20];
int stateCount=0;

int is_new_state(char s[3]){
    for(int i=0;i<stateCount;i++)
    {
        if(strcmp(states[i],s)==0)
            return 0;
    }
    return 1;
}

void add_state_result(char s[3],int i)
{
    for(int j=0;j<i;j++)
    {
        if(strcmp(result[j],s)==0)
            return;
    }
    strcpy(result[i],s);
}

void display(int n) {
    printf("\nEpsilon closure of %s = {", copy);
    for (int k = 0; k < n; k++) {
        printf(" %s", result[k]);
    }
    printf(" }\n");
}

void compute_epsilon_closure(char start[3], FILE *fp) {
    char stack[20][3];
    int top = 0;
    int i = 0;

    strcpy(stack[top++], start);
    strcpy(result[i++], start);

    char st1[3], input[3], st2[3];

    while (top > 0) {
```

```

    top--;
    char current[3];
    strcpy(current, stack[top]);

    rewind(fp);

    while (fscanf(fp, "%s %s %s", st1, input, st2) != EOF) {
        if (strcmp(st1, current) == 0 && strcmp(input, "e") == 0) {
            int already = 0;
            for (int j = 0; j < i; j++) {
                if (strcmp(result[j], st2) == 0) {
                    already = 1;
                    break;
                }
            }

            if (!already) {
                strcpy(stack[top++], st2);
                strcpy(result[i++], st2);
            }
        }
    }

    display(i);
}

int main() {
    FILE *fp = fopen("epsilon.txt", "r");
    if (!fp) {
        printf("Error: Could not open epsilon.txt\n");
        return 1;
    }

    char state1[3], input[3], state2[3];

    while (fscanf(fp, "%s %s %s", state1, input, state2) != EOF) {
        if (is_new_state(state1)) {
            strcpy(states[stateCount++], state1);
        }
        if (is_new_state(state2)) {
            strcpy(states[stateCount++], state2);
        }
    }
}

```

```
for (int k = 0; k < stateCount; k++) {  
    strcpy(copy, states[k]);  
    compute_epsilon_closure(states[k], fp);  
}  
  
fclose(fp);  
return 0;  
}
```

output:

Epsilon closure of A = { A B C }

Epsilon closure of B = { B C }

Epsilon closure of C = { C }

3)EPSILON NFA TO NFA

```
#include <stdio.h>
#include <string.h>

char states[20][3], inputSymbols[10][3], transitions[50][3][3];
int stateCount = 0, inputCount = 0, transCount = 0;

char result[50][3];
int resCount = 0;

int is_new_state(char s[3]) {
    for (int i = 0; i < stateCount; i++)
        if (strcmp(states[i], s) == 0) return 0;
    return 1;
}

int is_new_input(char s[3]) {
    for (int i = 0; i < inputCount; i++)
        if (strcmp(inputSymbols[i], s) == 0) return 0;
    return 1;
}

void add_result(char s[3]) {
    for (int i = 0; i < resCount; i++)
        if (strcmp(result[i], s) == 0) return;
    strcpy(result[resCount++], s);
}

void reset_result() {
    resCount = 0;
}

void compute_epsilon_closure(char start[3]) {
    char stack[20][3];
    int top = 0;
    strcpy(stack[top++], start);
    add_result(start);

    while (top > 0) {
        char current[3];
        strcpy(current, stack[--top]);
```



```

for (int i = 0; i < transCount; i++) {
    if (strcmp(transitions[i][0], current) == 0 &&
        strcmp(transitions[i][1], "e") == 0) {

        int already = 0;
        for (int j = 0; j < resCount; j++) {
            if (strcmp(result[j], transitions[i][2]) == 0) {
                already = 1;
                break;
            }
        }

        if (!already) {
            add_result(transitions[i][2]);
            strcpy(stack[top++], transitions[i][2]);
        }
    }
}
}

```

```

void print_epsilon_closures() {
    printf("\nEpsilon Closures:\n");
    for (int s = 0; s < stateCount; s++) {
        reset_result();
        compute_epsilon_closure(states[s]);

        printf("ε-closure(%s) = { ", states[s]);
        for (int i = 0; i < resCount; i++) {
            printf("%s ", result[i]);
        }
        printf("}\n");
    }
}

```

```

void compute_move(char from[3], char symbol[3]) {
    for (int i = 0; i < transCount; i++) {
        if (strcmp(transitions[i][0], from) == 0 &&
            strcmp(transitions[i][1], symbol) == 0) {
            add_result(transitions[i][2]);
        }
    }
}

```

```

void convert_to_NFA_without_e() {

```

```

printf("\nNFA Without Epsilon Transitions ( $\delta$ ): \n");
for (int s = 0; s < stateCount; s++) {
    for (int i = 0; i < inputCount; i++) {
        reset_result();
        compute_epsilon_closure(states[s]);

        char eclosure[20][3];
        int ecount = resCount;
        for (int k = 0; k < resCount; k++)
            strcpy(eclosure[k], result[k]);

        reset_result();
        for (int k = 0; k < ecount; k++)
            compute_move(eclosure[k], inputSymbols[i]);

        char moveResult[20][3];
        int mcount = resCount;
        for (int k = 0; k < resCount; k++)
            strcpy(moveResult[k], result[k]);

        reset_result();
        for (int k = 0; k < mcount; k++)
            compute_epsilon_closure(moveResult[k]);

        if (resCount > 0) {
            printf("\delta'(%s, %s) = { ", states[s], inputSymbols[i]);
            for (int j = 0; j < resCount; j++)
                printf("%s ", result[j]);
            printf("}\n");
        }
    }
}

int main() {
    FILE *fp = fopen("epsilon.txt", "r");
    if (!fp) {
        printf("Error: Could not open epsilon.txt\n");
        return 1;
    }

    char state1[3], input[3], state2[3];
    while (fscanf(fp, "%s %s %s", state1, input, state2) != EOF) {
        strcpy(transitions[transCount][0], state1);
        strcpy(transitions[transCount][1], input);
    }
}

```

```

strcpy(transitions[transCount][2], state2);
transCount++;

if (is_new_state(state1)) strcpy(states[stateCount++], state1);
if (is_new_state(state2)) strcpy(states[stateCount++], state2);

if (strcmp(input, "e") != 0 && is_new_input(input))
    strcpy(inputSymbols[inputCount++], input);
}

print_epsilon_closures();
convert_to_NFA_without_e();

fclose(fp);
return 0;
}

```

output:

Epsilon Closures:

ϵ -closure(A) = { A B C }

ϵ -closure(B) = { B C }

ϵ -closure(C) = { C }

NFA Without Epsilon Transitions (δ'):

$\delta'(A, a) = \{ A B C \}$

$\delta'(A, b) = \{ B C \}$

$\delta'(A, c) = \{ C \}$

$\delta'(B, b) = \{ B C \}$

$\delta'(B, c) = \{ C \}$

$\delta'(C, c) = \{ C \}$

4) NFA to DFA

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char states[10][10], symbols[10][10];
int nfa_count = 0, symbol_count = 0;

char nfa_trans[10][10][10][10]; // [state][symbol][next_state]
int nfa_trans_count[10][10]; // Count of transitions for each state-symbol

char dfa_states[100][100];
int dfa_count = 0;

int dfa_trans[100][10]; // Transition table

int is_new_state(char *state) {
    for (int i = 0; i < dfa_count; i++) {
        if (strcmp(dfa_states[i], state) == 0)
            return 0;
    }
    return 1;
}

void add_dfa_state(char *state) {
    strcpy(dfa_states[dfa_count++], state);
}

int get_state_index(char *state) {
    for (int i = 0; i < nfa_count; i++) {
        if (strcmp(states[i], state) == 0)
            return i;
    }
    return -1;
}

int get_symbol_index(char *symbol) {
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbols[i], symbol) == 0)
            return i;
    }
    return -1;
}
```

```

int get_dfa_index(char *state) {
    for (int i = 0; i < dfa_count; i++) {
        if (strcmp(dfa_states[i], state) == 0)
            return i;
    }
    return -1;
}

void split_and_transition(char *composite, char *symbol, char *result) {
    char temp_states[10][10];
    int temp_count = 0;
    result[0] = '\0';

    int sym_index = get_symbol_index(symbol);
    if (sym_index == -1) return;

    // Parse composite state (split by comma)
    char temp[100];
    strcpy(temp, composite);
    char *token = strtok(temp, ",");

    while (token) {
        int s = get_state_index(token);

        if (s != -1) {
            // Add all transitions from this state on this symbol
            for (int i = 0; i < nfa_trans_count[s][sym_index]; i++) {
                // Check if this state is already in our result
                int found = 0;
                for (int j = 0; j < temp_count; j++) {
                    if (strcmp(temp_states[j], nfa_trans[s][sym_index][i]) == 0) {
                        found = 1;
                        break;
                    }
                }
                if (!found) {
                    strcpy(temp_states[temp_count++], nfa_trans[s][sym_index][i]);
                }
            }
        }
        token = strtok(NULL, ",");
    }

    // Sort the states alphabetically
    for (int i = 0; i < temp_count - 1; i++) {

```

```

    for (int j = i + 1; j < temp_count; j++) {
        if (strcmp(temp_states[i], temp_states[j]) > 0) {
            char temp_swap[10];
            strcpy(temp_swap, temp_states[i]);
            strcpy(temp_states[i], temp_states[j]);
            strcpy(temp_states[j], temp_swap);
        }
    }
}

// Create result string
for (int i = 0; i < temp_count; i++) {
    strcat(result, temp_states[i]);
    if (i < temp_count - 1)
        strcat(result, ",");
}
}

void print_nfa() {
    printf("NFA Transitions:\n");
    for (int i = 0; i < nfa_count; i++) {
        for (int j = 0; j < symbol_count; j++) {
            for (int k = 0; k < nfa_trans_count[i][j]; k++) {
                printf("%s %s %s\n", states[i], symbols[j], nfa_trans[i][j][k]);
            }
        }
    }
    printf("\n");
}

void convert_nfa_to_dfa() {
    // Initialize DFA transition table
    for (int i = 0; i < 100; i++) {
        for (int j = 0; j < 10; j++) {
            dfa_trans[i][j] = -1;
        }
    }
}

add_dfa_state(states[0]); // Start with initial NFA state

int i = 0;
while (i < dfa_count) {
    for (int j = 0; j < symbol_count; j++) {
        char transition_result[100] = "";

```

```

        split_and_transition(dfa_states[i], symbols[j], transition_result);

        if (strlen(transition_result) == 0) {
            dfa_trans[i][j] = -1;
        } else {
            if (is_new_state(transition_result))
                add_dfa_state(transition_result);

            dfa_trans[i][j] = get_dfa_index(transition_result);
        }
    }
    i++;
}

void print_dfa_table() {
    printf("\nDFA Transition Table:\n\n");
    printf("DFA State\t\t");
    for (int i = 0; i < symbol_count; i++) {
        printf("%s\t", symbols[i]);
    }
    printf("\n");

    for (int i = 0; i < dfa_count; i++) {
        printf("%s\t\t\t", dfa_states[i]);
        for (int j = 0; j < symbol_count; j++) {
            if (dfa_trans[i][j] == -1)
                printf("-\t");
            else
                printf("%s\t", dfa_states[dfa_trans[i][j]]);
        }
        printf("\n");
    }
}

int main() {
    // Initialize arrays
    for (int i = 0; i < 10; i++) {
        for (int j = 0; j < 10; j++) {
            nfa_trans_count[i][j] = 0;
        }
    }

    FILE *fp = fopen("nfa.txt", "r");
    if (!fp) {

```

```

    printf("Error: Could not open nfa.txt\n");
    return 1;
}

char from[10], symbol[10], to[10];

while (fscanf(fp, "%s %s %s", from, symbol, to) != EOF) {
    int s_index = get_state_index(from);
    if (s_index == -1) {
        strcpy(states[nfa_count], from);
        s_index = nfa_count++;
    }

    int t_index = get_state_index(to);
    if (t_index == -1) {
        strcpy(states[nfa_count], to);
        t_index = nfa_count++;
    }

    int sym_index = get_symbol_index(symbol);
    if (sym_index == -1) {
        strcpy(symbols[symbol_count], symbol);
        sym_index = symbol_count++;
    }

    strcpy(nfa_trans[s_index][sym_index][nfa_trans_count[s_index][sym_index]+
+], to);
}

fclose(fp);

printf("Input NFA:\n");
print_nfa();

convert_nfa_to_dfa();
print_dfa_table();

return 0;
}

```

output:

Input NFA:
NFA Transitions:

q0 0 q0
q0 0 q1
q1 1 q1
q1 1 q2
q2 0 q0

DFA Transition Table:

DFA State	0	1
q0	q0,q1	-
q0,q1	q0,q1	q1,q2
q1,q2	q0	q1,q2

5) MINIMIZED DFA

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char states[100][100], symbols[10][10];
int state_count = 0, symbol_count = 0;
int dfa_trans[100][10];
int final_states[100];
int is_final[100];

int partition[100];
int partition_count = 0;
char partition_states[100][100][100];
int partition_size[100];

char min_states[100][100];
int min_trans[100][10];
int min_state_count = 0;
int min_final[100];

int get_state_index(char *state) {
    for (int i = 0; i < state_count; i++) {
        if (strcmp(states[i], state) == 0)
            return i;
    }
    return -1;
}

int get_symbol_index(char *symbol) {
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbols[i], symbol) == 0)
            return i;
    }
    return -1;
}

void print_dfa() {
    printf("Original DFA:\n");
    printf("State\t\t");
    for (int i = 0; i < symbol_count; i++) {
```

```

    printf("%s\t", symbols[i]);
}
printf("Final\n");

for (int i = 0; i < state_count; i++) {
    printf("%s\t\t\t", states[i]);
    for (int j = 0; j < symbol_count; j++) {
        if (dfa_trans[i][j] == -1)
            printf("-\t");
        else
            printf("%s\t", states[dfa_trans[i][j]]);
    }
    printf("%s\n", is_final[i] ? "YES" : "NO");
}
printf("\n");
}

void initialize_partitions() {

    partition_count = 0;

    partition_size[0] = 0;
    for (int i = 0; i < state_count; i++) {
        if (!is_final[i]) {
            strcpy(partition_states[0][partition_size[0]], states[i]);
            partition[i] = 0;
            partition_size[0]++;
        }
    }
    if (partition_size[0] > 0) partition_count++;

    partition_size[1] = 0;
    for (int i = 0; i < state_count; i++) {
        if (is_final[i]) {
            strcpy(partition_states[1][partition_size[1]], states[i]);
            partition[i] = 1;
            partition_size[1]++;
        }
    }
    if (partition_size[1] > 0) partition_count++;
}

int get_partition_of_state(int state_index) {

```

```

    return partition[state_index];
}

int are_states_distinguishable(int state1, int state2) {

    for (int sym = 0; sym < symbol_count; sym++) {
        int next1 = dfa_trans[state1][sym];
        int next2 = dfa_trans[state2][sym];

        if (next1 == -1 && next2 == -1) continue;
        if (next1 == -1 || next2 == -1) return 1;

        if (get_partition_of_state(next1) != get_partition_of_state(next2)) {
            return 1;
        }
    }
    return 0;
}

void refine_partitions() {
    int changed = 1;

    while (changed) {
        changed = 0;

        for (int p = 0; p < partition_count; p++) {
            if (partition_size[p] <= 1) continue;

            int new_partition[100];
            int new_partition_size = 0;

            for (int i = 1; i < partition_size[p]; i++) {
                int state1_idx = get_state_index(partition_states[p][0]);
                int state2_idx = get_state_index(partition_states[p][i]);

                if (are_states_distinguishable(state1_idx, state2_idx)) {

                    new_partition[new_partition_size++] = state2_idx;
                    changed = 1;
                }
            }
        }

        if (new_partition_size > 0) {

```

```

partition_size[partition_count] = new_partition_size;
for (int i = 0; i < new_partition_size; i++) {
    strcpy(partition_states[partition_count][i], states[new_partition[i]]);
    partition[new_partition[i]] = partition_count;
}

```

```

int new_size = 0;
for (int i = 0; i < partition_size[p]; i++) {
    int state_idx = get_state_index(partition_states[p][i]);
    int found = 0;
    for (int j = 0; j < new_partition_size; j++) {
        if (new_partition[j] == state_idx) {
            found = 1;
            break;
        }
    }
    if (!found) {
        strcpy(partition_states[p][new_size++], partition_states[p][i]);
    }
}
partition_size[p] = new_size;
partition_count++;
}
}
}
}
}

```

```

void create_minimized_dfa() {
    min_state_count = partition_count;

```

```

for (int i = 0; i < partition_count; i++) {
    strcpy(min_states[i], "{}");
    for (int j = 0; j < partition_size[i]; j++) {
        strcat(min_states[i], partition_states[i][j]);
        if (j < partition_size[i] - 1) strcat(min_states[i], ",");
    }
    strcat(min_states[i], "}");
}

```

```

min_final[i] = 0;
for (int j = 0; j < partition_size[i]; j++) {
    int state_idx = get_state_index(partition_states[i][j]);
    if (is_final[state_idx]) {

```

```

        min_final[i] = 1;
        break;
    }
}
}

for (int i = 0; i < partition_count; i++) {
    for (int j = 0; j < symbol_count; j++) {

        int rep_state_idx = get_state_index(partition_states[i][0]);
        int next_state = dfa_trans[rep_state_idx][j];

        if (next_state == -1) {
            min_trans[i][j] = -1;
        } else {
            min_trans[i][j] = get_partition_of_state(next_state);
        }
    }
}
}
}

```

```

void print_minimized_dfa() {
    printf("Minimized DFA:\n");
    printf("State\t\t\t");
    for (int i = 0; i < symbol_count; i++) {
        printf("%s\t", symbols[i]);
    }
    printf("Final\n");

    for (int i = 0; i < min_state_count; i++) {
        printf("%s\t\t", min_states[i]);
        for (int j = 0; j < symbol_count; j++) {
            if (min_trans[i][j] == -1)
                printf("-\t\t");
            else
                printf("%s\t", min_states[min_trans[i][j]]);
        }
        printf("%s\n", min_final[i] ? "YES" : "NO");
    }
}

```

```

void print_partitions() {
    printf("Final Partitions:\n");
    for (int i = 0; i < partition_count; i++) {

```

```

    printf("Partition %d: {" , i);
    for (int j = 0; j < partition_size[i]; j++) {
        printf("%s", partition_states[i][j]);
        if (j < partition_size[i] - 1) printf(", ");
    }
    printf("}\n");
}
printf("\n");
}

```

```

int main() {
    FILE *fp = fopen("dfa.txt", "r");
    if (!fp) {
        printf("Error: Could not open dfa.txt\n");
        printf("Format: state symbol next_state\n");
        printf("Final states should be marked with 'F' after the state name\n");
        return 1;
    }
}

```

```

for (int i = 0; i < 100; i++) {
    is_final[i] = 0;
    for (int j = 0; j < 10; j++) {
        dfa_trans[i][j] = -1;
    }
}

```

```

char from[100], symbol[10], to[100];

```

```

while (fscanf(fp, "%s %s %s", from, symbol, to) != EOF) {

```

```

    int from_final = 0, to_final = 0;
    char from_clean[100], to_clean[100];

```

```

    if (from[strlen(from)-1] == 'F') {
        from_final = 1;
        strncpy(from_clean, from, strlen(from)-1);
        from_clean[strlen(from)-1] = '\0';
    } else {
        strcpy(from_clean, from);
    }

```

```

    if (to[strlen(to)-1] == 'F') {
        to_final = 1;
        strncpy(to_clean, to, strlen(to)-1);
    }

```

```
    to_clean[strlen(to)-1] = '\0';  
} else {  
    strcpy(to_clean, to);  
}
```

```
int from_idx = get_state_index(from_clean);  
if (from_idx == -1) {  
    strcpy(states[state_count], from_clean);  
    from_idx = state_count++;  
}
```

```
int to_idx = get_state_index(to_clean);  
if (to_idx == -1) {  
    strcpy(states[state_count], to_clean);  
    to_idx = state_count++;  
}
```

```
if (from_final) is_final[from_idx] = 1;  
if (to_final) is_final[to_idx] = 1;
```

```
int sym_idx = get_symbol_index(symbol);  
if (sym_idx == -1) {  
    strcpy(symbols[symbol_count], symbol);  
    sym_idx = symbol_count++;  
}
```

```
    dfa_trans[from_idx][sym_idx] = to_idx;  
}
```

```
fclose(fp);
```

```
print_dfa();
```

```
initialize_partitions();  
refine_partitions();
```

```
print_partitions();
```

```
create_minimized_dfa();  
print_minimized_dfa();
```



```
    return 0;
}
```

output:

Original DFA:

State	0	1	Final
q0		q1	q3 NO
q1		q2	q4 NO
q3		q2	q4 NO
q2		q1	q4 NO
q4		q4	q4 YES

Final Partitions:

Partition 0: {q0}

Partition 1: {q4}

Partition 2: {q1, q3, q2}

Minimized DFA:

State	0	1	Final
{q0}	{q1,q3,q2}	{q1,q3,q2}	NO
{q4}	{q4}	{q4}	YES
{q1,q3,q2}	{q1,q3,q2}	{q4}	NO